N-Body Simulation

Using OpenCL on an AMD iGPU

Nagy Levente

April 26, 2026

Eötvös Lóránd University

Contents

1 Introduction **2**
1.1 N-Body simulation . . . 2

2 Problem **2**
2.1 General N-Body problem . . . 2
2.2 N-Body computing . . . 2
2.3 Barnes-Hut alogithm . . . 2
2.3.1 Idea . . . 2
2.3.2 Problem with the algorithm on GPU . . . 3

3 Solution **3**
3.1 Steps . . . 3
3.2 Data Structure . . . 3
3.3 Buffers . . . 4
3.4 Kernels . . . 5
3.4.1 Bounding Box Kernel . . . 5
3.4.2 Build Tree Kernel . . . 7
3.4.3 Summarize Tree Kernel . . . 9
3.4.4 Sort Kernel . . . 11
3.4.5 Force Kernel . . . 12
3.4.6 Integrate Kernel . . . 14
3.4.7 Write Position Kernel . . . 16
3.5 Host Side . . . 16
3.5.1 Simulation Class . . . 17
3.5.2 Render . . . 20
3.6 Shaders . . . 22
3.6.1 Vertex Shader . . . 22
3.6.2 Fragment Shader . . . 23

4 Performance Analysis **25**
4.1 Methodology . . . 25
4.2 Results . . . 25
4.3 Analysis . . . 25
4.3.1 Two Distinct Regimes . . . 25
4.3.2 Hardware Observations . . . 27
4.3.3 Real-Time Threshold . . . 27
4.4 Summary . . . 28

1 Introduction

1.1 N-Body simulation

An N-body simulation models the motion of **N** particles in space, where each particle is influenced by physical forces such as gravity, defined by the gravitational constant **G**.

2 Problem

2.1 General N-Body problem

The general N-body problem has no closed-form analytical solution for N ¿= 3 (the famous three-body problem).

2.2 N-Body computing

Computing the forces from each body acting on every other body is costly when **N** is a big number. A simple brute force algorithm time complexity is

$$O(n^2)$$

For a better algorithm there exists the Barnes-Hut algorithm which time complexity is

$$O(n \log(n))$$

So its a much better solution for computing the forces. We can plot them to see the difference

2.3 Barnes-Hut alogithm

2.3.1 Idea

The Barnes-Hut algorithm divides the space where the bodies located into 8 smaller chunks if there is more than 1 body is in a chunk. After recursively doing this for every body we are left with an Octree (each parent can have up to 8 children). In the given tree each node represtens a 3Dimensional space with a body. Than we compute the **COM** (Center Of Mass) of each node and use this formula

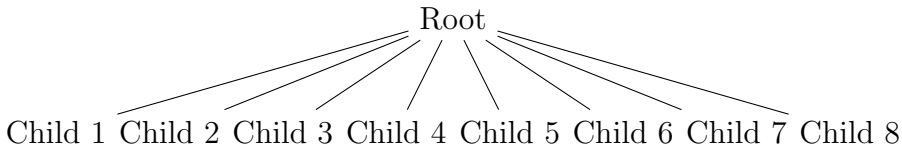
$$\frac{s}{d} < \theta$$

where:

- s is the size of the region
- d is the distance to the center of mass
- θ controls the trade-off between accuracy and performance

To determine if 2 nodes of the tree can be used as 1, reducing the computeitation steps we have to do, reducing the time complexity to just

$$O(n \log(n))$$



2.3.2 Problem with the algorithm on GPU

The Barnes-Hut algorithm relies on tree traversal, which is naturally recursive. However, recursion is inefficient or unsupported in GPU programming models such as OpenCL. Therefore, the algorithm must be reformulated using iterative techniques and explicit memory structures to work.

3 Solution

This section is going to describe the solution and the implementation of the Barnes-Hut alogithm on the GPU

3.1 Steps

1. Computing the bounding box containing all of the bodies
2. Build the octree and instert the bodies into it
3. Summerize the information of the bodies (Center Of Mass)
4. Sorting the bodies based on spatial distance (not necessary but helps with the performance)
5. Calculating the force
6. Applying the force to each body, updating position and velocities
7. Passing the global data to the pipline

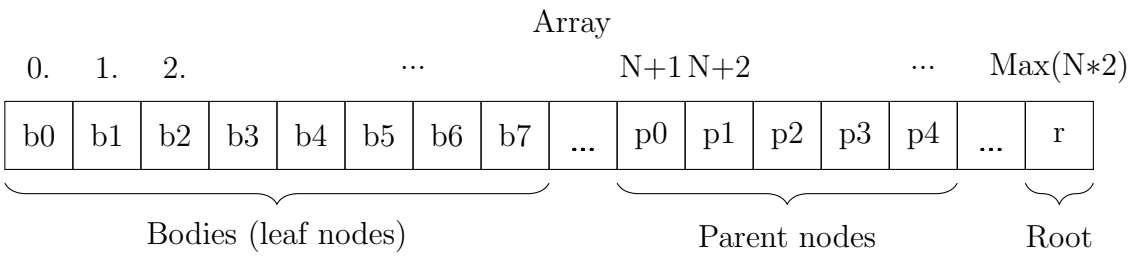
3.2 Data Structure

Efficient data representation is critical for achieving high performance on the GPU. Instead of using a traditional pointer-based tree structure, the simulation uses a **linearized octree** stored in contiguous memory buffers.

Overview All simulation data is stored in flat arrays in global memory. Each index represents either a body or an internal node of the octree.

- The first N entries correspond to physical bodies (leaf nodes)
- The remaining entries represent internal octree nodes (parents and root)

The size of the array is chosen to be approximately $2N$, which provides a safe upper bound. In an octree, each body corresponds to at most one leaf node, and the number of internal nodes is strictly less than the number of bodies. Therefore, the total number of nodes (bodies plus internal nodes) never exceeds $2N$.



3.3 Buffers

Position and Mass Storage The positions and masses are stored in separate arrays:

- `pos_x, pos_y, pos_z` — coordinates of bodies and nodes
- `mass` — mass of each body or aggregated node

For internal nodes, the position represents the **center of mass (COM)**.

Tree Structure Representation The octree structure is encoded using index-based arrays:

- `child` — stores indices of the 8 children for each node
- `start` — marks the beginning of a node's body range

Each node has exactly 8 child slots:

$$\texttt{child[node * 8 + i]}, \quad i \in [0, 7]$$

Special values are used to indicate state:

- `EMPTY (-1)` — no child present
- `LOCKED (-2)` — node is being modified by another thread

Node Properties Additional arrays store per-node metadata:

- `nodeSize` — spatial size (half-width) of the node
- `nodeCount` — number of bodies contained in the node

The root node is stored at a fixed index (`NUMBER_OF_NODES`) and represents the bounding volume of the entire system.

- `NUMBER_OF_NODES (numNodes)` — Size of the flat array $\rightarrow 2N$

Auxiliary Buffers Several additional buffers are used during computation:

- `sorted` — stores bodies ordered by spatial distance
- `accX, accY, accZ` — accumulated forces
- `vx, vy, vz` — velocities of bodies

Advantages of the Approach This data-oriented design has several advantages:

- Avoids dynamic memory allocation and recursion
- Enables parallel processing across threads
- Improves cache efficiency and memory coalescing
- Allows interoperability with OpenGL buffers for rendering

Summary By representing the octree as a set of flat arrays, the simulation achieves a structure that is well-suited for GPU execution while still supporting efficient Barnes-Hut traversal.

3.4 Kernels

3.4.1 Bounding Box Kernel

The purpose of this kernel is to compute the global bounding box that contains all bodies in the simulation. This bounding box is later used to initialize the root node of the octree structure required by the Barnes-Hut algorithm.

Overview The kernel performs a parallel reduction to determine the minimum and maximum coordinates along each axis (x, y, z). Each work-group processes a subset of the bodies and computes a local bounding box, which is then combined into a global result.

Step 1: Local Reduction Each work-item iterates over the input data using a strided access pattern:

$$i = \text{localId} + \text{groupId} \cdot \text{localSize}$$

This ensures that all bodies are covered efficiently across all work-groups.
For each assigned body, the thread updates its local minimum and maximum values:

- $x_{\min}, x_{\max}$
- $y_{\min}, y_{\max}$
- $z_{\min}, z_{\max}$

These values are stored in shared local memory to allow fast intra-group communication.

Step 2: Parallel Reduction in Local Memory A tree-based reduction is performed within each work-group:

- At each step, half of the threads combine their values with a partner thread
- This continues until thread 0 holds the result for the entire work-group

This reduces the per-group bounding box to a single set of values.

Step 3: Global Reduction The first thread of each work-group writes its result into global memory. An atomic counter (`blockCount`) is used to detect when all work-groups have finished. The last work-group to finish performs a final reduction over all partial results to compute the global bounding box.

Step 4: Root Node Initialization Once the global bounding box is known, the kernel initializes the root node of the octree:

- The center of the root node is computed as:

$$\mathbf{c} = \frac{1}{2}(\mathbf{min} + \mathbf{max})$$

- The size (radius) of the node is:

$$r = \frac{1}{2} \max\left(x_{\max} - x_{\min}, y_{\max} - y_{\min}, z_{\max} - z_{\min}\right)$$

The root node is stored at index `NUMBER_OF_NODES` and initialized with:

- Position = center of the bounding box
- Size = computed radius
- Mass = -1 (indicating an internal node)
- Children = `EMPTY`

Synchronization The kernel uses:

- `barrier()` for synchronization within a work-group
- `atomic_inc()` to coordinate between work-groups

This ensures correct ordering despite the lack of global synchronization in OpenCL.

Result At the end of execution:

- The global bounding box is computed
- The root node of the octree is initialized
- The simulation is ready for tree construction

3.4.2 Build Tree Kernel

The purpose of this kernel is to construct the octree by inserting each body into the hierarchical spatial structure. This step transforms the flat list of bodies into a tree representation that enables efficient Barnes-Hut force computation.

Overview Each work-item is responsible for inserting one or more bodies into the tree. The insertion process starts at the root node and iteratively descends through the tree until a suitable location for the body is found.

The kernel avoids recursion by using an iterative traversal approach, which is required for efficient execution on the GPU.

Step 1: Initialization The root node is retrieved from the precomputed bounding box:

- Position: $(x_{\text{root}}, y_{\text{root}}, z_{\text{root}})$
- Size: `radius`

Each thread processes bodies in a strided manner:

$$\text{bodyIdx} += \text{localSize} \cdot \text{numGroups}$$

This ensures all bodies are distributed evenly across threads.

Step 2: Tree Traversal For each body, the algorithm determines which of the 8 octants it belongs to. This is encoded as a 3-bit index:

- Bit 0: x comparison
- Bit 1: y comparison
- Bit 2: z comparison

This produces a value in the range $[0, 7]$, identifying the child node.
The traversal continues as long as the selected child is an internal node:

- Update the current node
- Reduce the node size (half each level)
- Recompute the child index

Step 3: Concurrent Insertion When a leaf position is reached, the kernel attempts to insert the body. Since multiple threads may access the same node, atomic operations are required.

- `atom_cmpxchg()` is used to lock the target child slot
- `LOCKED (-2)` indicates that another thread is modifying the node

Three cases are possible:

1. **Empty slot:** The body is inserted directly.
2. **Occupied by another body:** A new internal node is created, and both bodies are redistributed into its children.
3. **Node already being modified:** The thread retries until the slot becomes available.

Step 4: Node Subdivision If two bodies fall into the same region, the space is subdivided:

- A new node is allocated using an atomic decrement on `bottom`
- The node's position and size are computed based on the parent
- The existing body is moved into the correct child of the new node

This process repeats until the bodies are separated or a stopping condition is reached.

Stopping Conditions To prevent infinite subdivision, two limits are enforced:

- Minimum cell size: `currentR < SOFTENING`
- Maximum depth: typically 64 levels

If either condition is met, the body is forcefully inserted without further subdivision.

Step 5: Depth Tracking Each thread tracks the maximum depth reached during insertion. At the end of execution, a global maximum is computed using:

- `atom_max(maxDepth, localMaxD)`

This value is useful for performance analysis and debugging.

Synchronization The kernel relies on:

- Atomic operations for safe concurrent updates
- Memory fences (`mem_fence`) to ensure visibility of writes

No global synchronization is required, allowing the kernel to scale efficiently.

Result After execution:

- All bodies are inserted into the octree
- Internal nodes are created where necessary
- The tree structure is fully constructed and ready for traversal

3.4.3 Summarize Tree Kernel

The purpose of this kernel is to compute aggregate information for each internal node of the octree. In particular, it calculates the **total mass** and **center of mass (COM)** for every node, which are essential for the Barnes-Hut force approximation.

Overview After the tree structure has been constructed, internal nodes do not yet contain valid physical data. This kernel traverses the tree bottom-up and computes:

- The total mass of each node
- The center of mass position
- The number of bodies contained in the node

Unlike other kernels, this step is executed by a **single work-item** (`global_id == 0`). This simplifies synchronization and guarantees correct ordering of computations.

Step 1: Iteration over Nodes The kernel iterates over all internal nodes:

```
for (node = bottom; node <= NUMBER_OF_NODES; node++)
```

- `bottom` marks the first internal node
- `NUMBER_OF_NODES` corresponds to the root node

Leaf nodes (bodies) are skipped since their mass and position are already known.

Step 2: Accumulating Child Information For each internal node, all 8 child slots are examined:

- If a child is `EMPTY`, it is ignored
- Otherwise, its contribution is accumulated

The following values are computed:

- Total mass:

$$M = \sum_i m_i$$

- Weighted position:

$$\mathbf{c} = \sum_i m_i \mathbf{x}_i$$

- Body count:

$$\text{count} = \begin{cases} 1 & \text{if child is a body} \\ \texttt{nodeCount[child]} & \text{if child is an internal node} \end{cases}$$

Step 3: Computing Center of Mass If the total mass is non-zero, the center of mass is computed as:

$$\mathbf{x}_{\text{COM}} = \frac{\sum_i m_i \mathbf{x}_i}{\sum_i m_i}$$

This value is written back into the node's position arrays (`x`, `y`, `z`).

Step 4: Updating Node Data Each node is updated with:

- `mass[node]` — total mass of the subtree
- `x[node], y[node], z[node]` — center of mass
- `nodeCount[node]` — number of bodies contained

A memory fence is used to ensure that writes are visible before continuing:

- `mem_fence(CLK_GLOBAL_MEM_FENCE)`

Design Choice This kernel is intentionally executed sequentially on a single thread:

- Ensures correct bottom-up dependency resolution
- Avoids complex synchronization between work-items
- The workload is relatively small compared to force computation

Although parallelization is possible, the overhead would outweigh the benefits for this stage of the algorithm.

Result After execution:

- All internal nodes contain valid mass and center of mass values
- Each node knows how many bodies it represents
- The octree is fully prepared for efficient Barnes-Hut force evaluation

3.4.4 Sort Kernel

The purpose of this kernel is to reorder bodies in memory according to their spatial locality, as defined by the octree structure. This step is not strictly required for correctness, but it significantly improves performance in later stages, especially during force computation, by improving memory coherence and cache efficiency.

Overview The kernel traverses the octree and produces a linear ordering of bodies in the array `sorted`. The ordering ensures that bodies that are close in space are also close in memory.

This improves:

- Memory coalescing on the GPU
- Cache utilization
- Traversal efficiency in the force computation step

Execution Model The kernel is executed by a single work-item:

- Only the thread with `global_id = 0` performs the computation
- This is sufficient since the operation is inherently sequential

Tree Traversal The traversal starts from the root node and proceeds downwards:

- Nodes are processed in reverse order, from the root toward the leaves
- The variable `btm` marks the lowest internal node in the tree

For each node:

- The starting index `s = start[cell]` determines where its data should be written
- If `start[cell] < 0`, the node is skipped

Processing Children Each node has up to 8 children. For each child:

- **Internal node ($c \geq$ NUM_BODIES):**
  - Assign its starting index:

    `start[c] = s`
  - Increment s by the number of bodies contained in that subtree:

    s += `nodeCount[c]`
- **Leaf node (body):**
  - Store the body index in the sorted array:

    `sorted[s++] = c`

Result After execution:

- The array `sorted` contains all bodies in a spatially coherent order
- The `start` array defines contiguous ranges for each subtree

Benefits This ordering provides several advantages:

- Bodies that are spatially close are processed together
- Memory accesses become more predictable and efficient
- Subsequent kernels (e.g., force computation) achieve better performance

Summary The sort kernel converts the hierarchical octree structure into a linear memory layout that preserves spatial locality. This transformation is key for achieving high performance on GPU architectures.

3.4.5 Force Kernel

The purpose of this kernel is to compute the gravitational acceleration acting on each body using the Barnes-Hut approximation. This is the most computationally intensive part of the simulation.

Overview Each work-item is responsible for computing the net force acting on a single body. Instead of evaluating all pairwise interactions ($O(N^2)$), the kernel traverses the octree and approximates distant groups of bodies as a single mass.

This reduces the complexity to approximately $O(N \log N)$.

Body Assignment Each thread processes one body:

- The index is obtained via `get_global_id(0)`
- The actual body index is retrieved from the `sorted` array

This ensures that bodies are processed in a spatially coherent order.

Initialization For each body:

- Its position (b_x, b_y, b_z) is loaded
- Accumulated acceleration (a_x, a_y, a_z) is initialized to zero

Tree Traversal The octree is traversed using an explicit stack:

- The root node is pushed onto the stack
- The traversal proceeds iteratively (no recursion)

At each step:

- A node is popped from the stack
- All of its children are examined

Distance Computation For each child node or body:

$$d^2 = (x_c - b_x)^2 + (y_c - b_y)^2 + (z_c - b_z)^2 + \epsilon^2$$

where:

- ϵ is the softening factor (`SOFTENING`)
- This prevents numerical instability at very small distances

The inverse distance is computed efficiently using:

$$\frac{1}{d} \approx \texttt{native_rsqrt}(d^2)$$

Interaction Cases Two cases are considered:

1. Leaf Node (Body) If the child represents a body:

- Self-interaction is skipped
- The gravitational force is computed directly:

$$F \propto \frac{G \cdot m}{d^3}$$

- The contribution is added to the acceleration

2. Internal Node If the child is an internal node, the Barnes-Hut criterion is applied:

$$\frac{s}{d} < \theta$$

where:

- s is the size of the node (`nodeSize`)
- d is the distance to the node's center of mass

- θ is the accuracy parameter
- If the condition holds:
  - The node is treated as a single mass
  - Its contribution is added directly
- Otherwise:
  - The node is expanded
  - It is pushed onto the stack for further traversal

Stack-Based Traversal The stack replaces recursion:

- Maximum size is fixed (128 entries)
- Ensures compatibility with GPU execution

This allows deep traversal while avoiding unsupported recursive calls.

Final Accumulation After traversal:

- The total acceleration is stored in:
  - `accX`, `accY`, `accZ`

Summary The force kernel performs an efficient approximation of gravitational interactions by:

- Traversing the octree iteratively
- Applying the Barnes-Hut approximation for distant nodes
- Computing exact forces for nearby bodies

This approach achieves a balance between accuracy and performance, making large-scale N-body simulations feasible on the GPU.

3.4.6 Integrate Kernel

The purpose of this kernel is to update the physical state of each body in the simulation by integrating velocity and position over time. This step applies the computed accelerations (from the force kernel) and advances the system forward using a simple numerical integration scheme.

Overview This kernel performs a basic **explicit Euler integration**, which updates:

- Velocity using acceleration
- Position using velocity

Although more advanced integrators (e.g. Leapfrog or Runge-Kutta) provide better numerical stability, Euler integration is chosen here due to its simplicity and low computational cost, which is suitable for real-time GPU execution.

Execution Model The kernel is executed in parallel, where each work-item processes multiple bodies using a strided loop:

- Each thread computes its starting index using `get_global_id(0)`
- Work is distributed using:

$$\texttt{stepSize} = \texttt{get_local_size(0)} \cdot \texttt{get_num_groups(0)}$$

This ensures full coverage of all bodies while maintaining balanced workload distribution across the GPU.

Step 1: Velocity Update The velocity of each body is updated based on the computed acceleration:

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \mathbf{a}(t) \cdot \Delta t$$

This step integrates acceleration into velocity, effectively applying Newton's second law.

Step 2: Position Update After updating velocity, positions are advanced using:

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \mathbf{v}(t) \cdot \Delta t$$

This step moves each body according to its current velocity.

Time Step Control The parameter `DT` represents the simulation time step:

- Smaller values improve numerical stability
- Larger values increase simulation speed but may reduce accuracy

Choosing an appropriate `DT` is critical for maintaining stable orbital motion.

Optional Constraints The kernel includes a commented condition:

- `// if (i == 0) continue;`

This can be used to freeze a special body (e.g. a "black hole" or fixed reference object), preventing it from being affected by forces or motion updates.

Memory Access Pattern All arrays (`x, y, z, vx, vy, vz, accX, accY, accZ`) are accessed sequentially per body index.

This design ensures:

- Coalesced global memory access on GPU
- Minimal cache misses
- High throughput during integration

Complexity The kernel has linear complexity:

$$O(N)$$

where N is the number of bodies. Each body is processed exactly once per simulation step.

Summary The integrate kernel advances the simulation state by:

- Applying computed accelerations to velocity
- Updating positions using velocity
- Performing a fully parallel, memory-efficient update

It represents the final stage of the simulation pipeline before rendering or the next physics iteration.

3.4.7 Write Position Kernel

The purpose of this kernel is to convert the position data from a structure-of-arrays (SoA) representation into an interleaved array format (AoS) that can be efficiently consumed by the rendering pipeline.

3.5 Host Side

The host-side implementation is divided into two main components:

- **Simulation** — responsible for computation and OpenCL interaction
- **SimulationRender** — responsible for visualization and user interaction

3.5.1 Simulation Class

The `Simulation` class is responsible for managing the entire compute pipeline. Its main responsibilities include:

- Initializing OpenCL context, kernels, and buffers
- Generating initial particle distributions
- Uploading data to the GPU
- Executing the simulation kernels each frame

Kernel Initialization All OpenCL kernels are stored in separate source files located in the `kernels` directory. These files are loaded and compiled at runtime by the `buildKernels` function.

During compilation, several parameters are passed as preprocessor definitions:

- `NUM_BODIES`
- `NUMBER_OF_NODES`
- `THREADS`, `WARPSIZE`

This approach allows the kernels to be specialized for the current simulation configuration, improving performance and flexibility.

Buffer Allocation The simulation uses a set of OpenCL buffers to store all relevant data:

- Positions, velocities, and accelerations
- Mass values
- Octree structure (children, node sizes, counts)

Buffers are allocated with sizes based on the number of bodies and nodes. All memory is allocated once and reused across simulation steps to avoid runtime overhead.

Generating Bodies The initial state of the simulation is generated on the host.

- The first body is treated as a central massive object ("black hole")
- Remaining bodies are assigned random masses within a predefined range

Different spatial distributions are supported:

- **Disk (default):** Bodies orbit around a central mass
- **Uniform:** Bodies are distributed uniformly in a cube

- **Sphere:** Bodies are distributed inside a sphere
- **Ring:** Bodies form a ring structure
- **Gaussian:** Normally distributed positions (flattened disk)
- **Plummer:** Astrophysically motivated density profile
- **NFW:** Dark matter halo distribution
- **Two Disk:** Two interacting galaxy-like systems

These distributions allow testing different physical scenarios and system behaviors.

Host-to-GPU Data Transfer After initialization, all generated data is uploaded to GPU memory using `enqueueWriteBuffer`. The buffers are then bound as arguments to the corresponding kernels.

This step ensures that all computation can be performed entirely on the GPU, minimizing CPU-GPU communication during the simulation loop.

Kernel Argument Setup Each kernel requires a specific set of input buffers and parameters. These are assigned once during initialization or updated when simulation parameters change.

Examples include:

- Physical constants (`G`, `DT`, `theta`)
- Buffer references (positions, velocities, tree structure)

Updating kernel arguments only when necessary reduces overhead during execution.

NDRange Configuration The execution configuration of each kernel is defined using OpenCL's NDRange model. The following ranges are used:

- `global = NDRange(NUM_BODIES)`
- `local = NDRange(THREADS)`

This configuration assigns one work-item per body for the majority of kernels, ensuring that the workload scales linearly with the number of particles.

Choice of Work-Group Size The local work size is fixed to `THREADS = 64`, which matches the native wavefront size of AMD GPUs. This ensures:

- Efficient utilization of SIMD execution units
- Minimal thread divergence within a wavefront
- Optimal scheduling by the GPU hardware

Using a work-group size aligned with the hardware execution width avoids underutilization and improves overall throughput.

Kernel-Specific Configurations Different kernels use slightly different NDRange configurations depending on their computational characteristics:

- **Data-parallel kernels** (bounding box, force, integrate, write):
  - Use `global = NUM_BODIES`
  - Each work-item processes one or more bodies
- **Tree construction kernel**:
  - Uses a smaller global range (`safe = THREADS`)
  - Each thread processes multiple bodies using strided access
  - This reduces contention on shared data structures (e.g. atomics)
- **Sequential kernels** (summarization, sorting):
  - Executed with a single work-group
  - In some cases, only one work-item performs the computation
  - This simplifies synchronization and avoids race conditions

Simulation Step Execution The core of the simulation is executed in the `step()` function, which is called once per frame.

Each step consists of the following stages:

1. Reset auxiliary buffers (accelerations, tree data, counters)
2. Compute bounding box
3. Build the octree structure
4. Compute center of mass for each node
5. Sort bodies for spatial locality
6. Compute gravitational forces
7. Integrate positions and velocities
8. Write positions to the OpenGL buffer

Between kernel launches, explicit synchronization is enforced using:

- `enqueueBarrierWithWaitList()`
- `queue.finish()`

This guarantees correct execution order despite the asynchronous nature of OpenCL.

OpenGL Interoperability To avoid unnecessary data transfers between CPU and GPU, the simulation uses OpenCL-OpenGL interoperability.

- Vertex Buffer Objects (VBOs) are shared between OpenCL and OpenGL
- The `writePositionsKernel` writes directly into these buffers

The workflow is as follows:

- Acquire OpenGL buffer (`enqueueAcquireGLObjects`)
- Execute kernel writing positions
- Release buffer back to OpenGL (`enqueueReleaseGLObjects`)

This approach eliminates redundant memory copies and significantly improves performance.

Dynamic Reconfiguration The simulation supports runtime parameter changes:

- **Restart:** Reinitializes buffers and distributions
- **Parameter update:** Updates kernel arguments without full reset

This enables interactive experimentation without restarting the application.

Summary The `Simulation` class orchestrates the entire GPU-based computation pipeline. It efficiently manages memory, kernel execution, and interoperability with OpenGL, providing a scalable and high-performance solution for real-time N-body simulation.

3.5.2 Render

The rendering system is implemented with OpenGL 2.1 and responsible for visualizing the the bodies and the physical motion applied to them.

Context and Window The pipeline initializes a GLFW Window and the OpenGL Context

The Window is created in full screen mode and can not be resized and vertical synchronization is disabled to allow maximum frame rate during simulation. Input callbacks are registered to handle:

- Keyboard input (exit and controls)
- Mouse movement (camera control)
- Scroll input (zooming)

Vertex Buffer Objects (VBOs) Two position buffers are created for particle data:

- Double-buffered position VBOs
- One mass VBO

The position buffers are shared with OpenCL using `cl::BufferGL`, allowing direct GPU-to-GPU communication without CPU involvement. Double buffering ensures that:

- One buffer is used for rendering
- The other buffer is written by OpenCL

This avoids synchronization stalls and improves performance.

Vertex Array Object (VAO) A VAO is used to define how vertex data is interpreted:

- Attribute 0: position (vec3)
- Attribute 1: mass (float)

This configuration allows the shader to access both spatial and physical properties of each particle.

Rendering Loop Each frame performs the following steps:

1. Clear the framebuffer
2. Compute projection and view matrices
3. Bind the active VBO (latest simulation data)
4. Bind the sprite texture
5. Issue draw call using `glDrawArrays(GL_POINTS)`

The number of rendered points corresponds to the number of simulated bodies.

OpenCL Interoperability The OpenGL buffers are shared with OpenCL using interoperability extensions.

The workflow per frame:

1. OpenCL acquires the OpenGL buffer
2. Kernel writes updated positions
3. OpenCL releases the buffer
4. OpenGL renders the updated data

This eliminates unnecessary memory transfers and ensures optimal performance.

Camera System The camera is implemented as an orbital system around a target point. It is defined by:

- Position (derived from yaw, pitch, and zoom)
- Target point
- Up vector

User interaction:

- Scroll: zoom in/out
- Drag: translate camera target
- Ctrl + Drag: rotate camera (yaw and pitch)

The view matrix is computed using `glm::lookAt`, while the projection matrix uses a perspective projection.

Graphical User Interface (ImGui) An interactive GUI is integrated using ImGui. It provides:

- Real-time parameter adjustment (gravity, timestep, etc.)
- Distribution selection
- Simulation reset controls
- Camera reset
- Performance monitoring (FPS)

The GUI is rendered after the main scene and does not interfere with simulation rendering.

3.6 Shaders

The rendering pipeline uses two programmable shaders written in GLSL:

- Vertex Shader
- Fragment Shader

3.6.1 Vertex Shader

The vertex shader is responsible for transforming particle positions and computing per-particle attributes that are later used for rendering.

Input Attributes Each particle provides:

- `inPos` – the 3D position of the particle
- `inMass` – the mass of the particle

Transformations The position is transformed from world space into clip space using the view and projection matrices:

- `view` – transforms world coordinates into camera space
- `projection` – applies perspective projection

Additionally, the distance from the camera is computed to allow distance-based scaling of particle size.

Derived Attributes Several normalized values are computed and passed to the fragment shader:

- `vMassNorm` – normalized logarithmic mass This maps a wide range of masses into $[0, 1]$, improving visual contrast.
- `vDistNorm` – normalized distance from the simulation center Used to simulate temperature variation based on position.
- `vIsBlackHole` – flag indicating whether the particle is a black hole Determined using the vertex ID.

Point Size Computation Particles are rendered as point sprites, and their size is determined dynamically:

- Black holes are assigned a large constant size for visibility
- Stars scale based on:
  - Their mass (larger mass $\rightarrow$ larger size)
  - Their distance from the camera (farther $\rightarrow$ smaller size)

This ensures that important structures remain visible while maintaining depth perception.

3.6.2 Fragment Shader

The fragment shader determines the final color and transparency of each particle.

Texture Sampling Each particle is rendered using a sprite texture:

- The texture is sampled using `gl_PointCoord`
- Fragments with very low alpha are discarded to create smooth circular particles

Star Color Model A simplified stellar temperature model is used to assign colors to particles.

Three base colors are defined:

- Cool stars – reddish tones
- Medium stars – yellow-white tones
- Hot stars – blue-white tones

A temperature value is computed as a weighted combination of:

- Mass contribution (`vMassNorm`)
- Distance contribution (`vDistNorm`)

This creates a gradient where:

- Massive or centrally located stars appear hotter (bluer)
- Smaller or distant stars appear cooler (redder)

The final color is obtained by interpolating between the predefined color ranges.

Final Color Composition The final fragment color is computed by combining:

- The sampled sprite texture
- A warm base tint
- The computed star color

Additive blending is used in the rendering pipeline, which results in:

- Bright regions where particles overlap
- A glowing, astrophysical appearance

Summary The shader system enhances the raw simulation data by:

- Mapping physical properties (mass, position) to visual attributes
- Dynamically scaling particle size
- Simulating stellar temperature through color interpolation

This approach enables an intuitive and visually rich representation of the simulated system while maintaining high rendering performance.

4 Performance Analysis

4.1 Methodology

Performance was measured by recording the average frames per second (FPS) reported by ImGui's internal frame time counter over a fixed observation window for each body count. All measurements were performed on the same hardware under identical simulation conditions:

- **GPU:** AMD Radeon gfx90c integrated GPU (7 compute units, 2048 MB LPDDR4)
- **Gravitational constant:** $G = 4.0$
- **Time step:** $\Delta t = 0.5$
- **Barnes-Hut parameter:** $\theta = 0.5$
- **Softening length:** $\varepsilon = 10.0$
- **Black hole mass:** $M_{BH} = 2 \times 10^9$
- **Distribution:** Disk
- **Thread count per work-group:** 64 (matching AMD wavefront size)

Body counts tested ranged from 64 to 65536 in powers of two.

4.2 Results

N (bodies)	Avg FPS	Avg ms/frame
64	373.9	2.67
128	313.3	3.19
256	298.0	3.36
512	256.8	3.89
1024	173.7	5.76
2048	113.0	8.85
4096	72.1	13.88
8192	42.8	23.34
16384	25.2	39.72
32768	13.4	74.77
65536	6.6	151.81

Table 1: Average performance for each body count

4.3 Analysis

4.3.1 Two Distinct Regimes

The data reveals two clearly distinct performance regimes, visible in Figure 1:

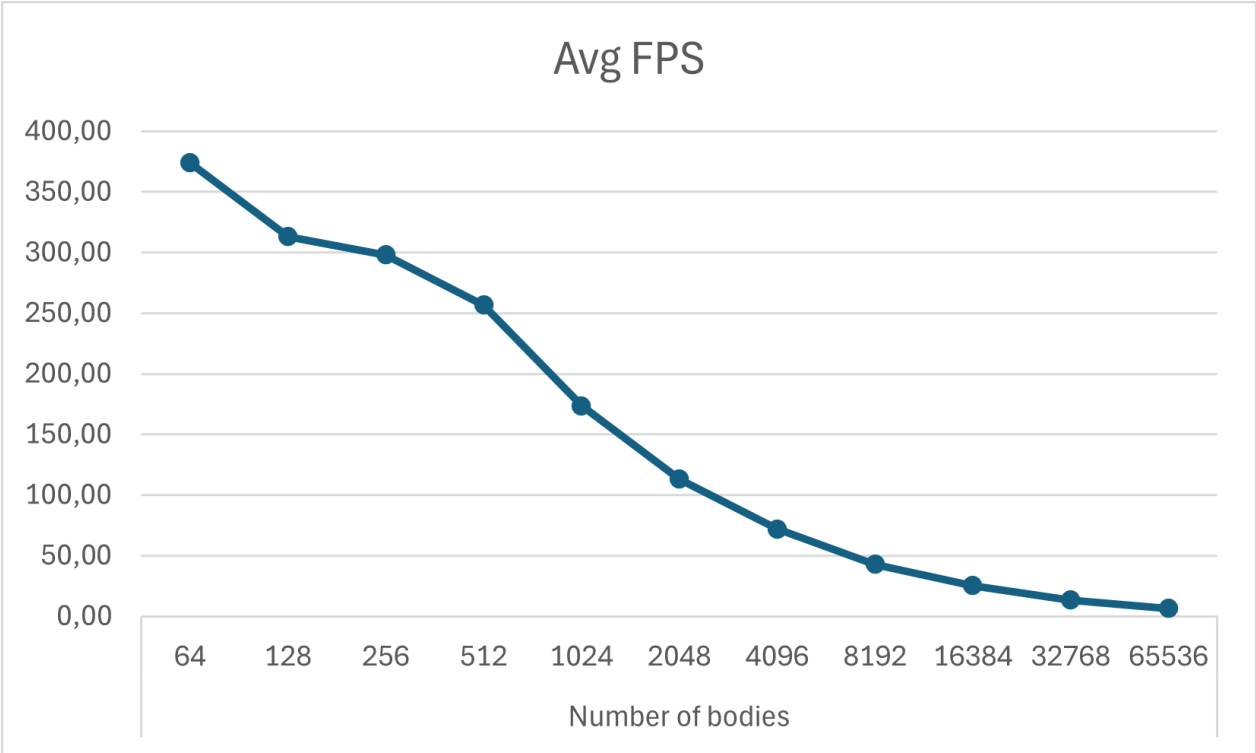


Figure 1: Average FPS as a function of the number of simulated bodies. The curve shows a rapid initial drop at small N followed by a smooth super-linear decline consistent with $O(N \log N)$ scaling.

Small N (64 – 512): fixed overhead dominated In this range FPS drops from 373 to 257 — a reduction of only 1.45× despite the body count increasing 8×. This indicates that per-frame work is dominated by **fixed overhead** that is independent of N:

- Tree buffer resets (`enqueueFillBuffer` over the full $2N$-node array)
- GPU kernel launch latency (7 kernels per frame)
- Single-threaded summarize and sort kernels traversing the full node array
- `queue.finish()` synchronization cost

A linear fit of frame time against $N \log_2 N$ for the full range yields an estimated fixed overhead of approximately **6.6** ms per frame, regardless of body count. At $N = 64$, this overhead accounts for the majority of the 2.67 ms total frame time.

Large N (1024 – 65536): $O(N \log N)$ dominated Above 1024 bodies the force kernel work becomes dominant and the $O(N \log N)$ scaling of the Barnes-Hut algorithm becomes clearly visible. The frame time fits closely to:

$$t_{\text{frame}} \approx 1.39 \times 10^{-4} \cdot N \log_2 N + 6.6 \text{ ms}$$

with residuals under 9% for all $N \geq 4096$. The table below shows the measured frame-time doubling ratio when N doubles, alongside the theoretical prediction:

N	**2N**	**Measured ratio**	$O(N \log N)$ **prediction**
4096	8192	1.68	1.92
8192	16384	1.70	1.94
16384	32768	1.88	1.96
32768	65536	2.03	1.97

Table 2: Frame-time doubling ratios when N is doubled. The measured ratio converges toward the theoretical $O(N \log N)$ prediction as N grows and fixed overhead becomes negligible.

As N grows large, the fixed overhead term becomes negligible and the ratio approaches the theoretical $O(N \log N)$ value. At $N = 65536$ the measured ratio of 2.03 is within 3% of the predicted 1.97, confirming that the force kernel dominates and scales as expected.

4.3.2 Hardware Observations

The measurements were taken on an integrated AMD Radeon gfx90c with 7 compute units sharing memory bandwidth with the CPU. Several hardware constraints affect the absolute numbers:

- **Memory bandwidth:** LPDDR4 shared with the CPU at 42 GB/s. At $N = 65536$, each force kernel traversal reads approximately $N \cdot \log_2 N \approx 1\,\mathrm{M}$ memory accesses per frame, stressing the bandwidth limit.
- **Compute units:** Only 7 CUs are available. The build-tree kernel is intentionally limited to `safe = THREADS = 64` total threads to prevent GPU Timeout Detection and Recovery (TDR) errors caused by atomic contention on shared tree nodes when too many threads compete simultaneously.
- **Thermal throttling:** As an integrated GPU with no dedicated cooling, sustained workloads at $N = 65536$ may reduce clock speeds, contributing to the slight super-linear increase in frame time observed at the high end.

4.3.3 Real-Time Threshold

A simulation is generally considered interactive if it runs at 30 FPS or above. Based on the measurements:

- Up to $\mathbf{N \approx 8000}$ bodies runs at 30+ FPS on this hardware
- $N = 8192$ achieves 42.8 FPS, comfortably above the threshold
- $N = 16384$ achieves 25.2 FPS, below 30 FPS but still visually smooth
- $N = 65536$ achieves 6.6 FPS, suitable for demonstration but not interactive in real time

On a discrete GPU with more compute units and dedicated memory bandwidth, the real-time threshold would shift significantly. The $O(N \log N)$ scaling means that a GPU with $10\times$ the compute throughput would push the real-time limit to approximately $N \approx 70000$ bodies.

4.4 Summary

The performance measurements confirm that the Barnes-Hut implementation achieves its expected $O(N \log N)$ time complexity for large particle counts. The fixed overhead of approximately 6.6 ms per frame, which includes kernel launches, buffer resets, and synchronization, limits performance for small simulations but becomes negligible as N grows.

The simulation maintains real-time performance (above 30 FPS) for up to approximately 8000 bodies on the tested AMD integrated GPU, and continues to run at visually useful frame rates up to 65536 bodies.